LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 2

EXPLORATORY DATA ANALYSIS DAN DATA PREPROCESSING



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Mahasiswa diharapkan mampu memahami konsep dasar Artificial Intelligence (AI) dan Machine Learning (ML).
2. Mahasiswa diharapkan mampu mengetahui dasar-dasar Machine Learning (ML)
3. Mahasiswa diharapkan mampu memahami konsep dasar Data Mining dan Data Preprocessing.

B Dasar Teori

B.1 Artificial Intelligence (AI)

Artificial Intelligence adalah teknologi yang memungkinkan mesin atau komputer untuk meniru kemampuan kognitif manusia. AI atau kecerdasan buatan adalah sistem yang dapat memahami, belajar, dan berpikir layaknya manusia dalam beberapa aspek, seperti mengenali pola, membuat prediksi, dan menyelesaikan masalah kompleks. Penggunaan AI sudah sangat luas dalam berbagai bidang, termasuk bisnis, kesehatan, dan hiburan.

AI juga mencakup berbagai teknologi, seperti machine learning, deep learning, dan natural language processing (NLP). Machine learning memungkinkan komputer belajar dari data dan pengalaman tanpa perlu diprogram secara eksplisit. Sementara itu, NLP digunakan untuk memungkinkan mesin memahami bahasa manusia, seperti yang kita temui pada asisten virtual seperti Siri dan Google Assistant [1].

B.2 Machine Learning (ML)

Teknologi machine learning (ML) adalah mesin yang dikembangkan untuk bisa belajar dengan sendirinya tanpa arahan dari penggunanya. Pembelajaran mesin dikembangkan berdasarkan disiplin ilmu lainnya seperti statistika, matematika dan

data mining sehingga mesin dapat belajar dengan menganalisis data tanpa perlu di program ulang atau diperintah.

Dalam hal ini machine learning memiliki kemampuan untuk memperoleh data yang ada dengan perintah ia sendiri. ML juga dapat mempelajari data yang ada dan data yang ia peroleh sehingga bisa melakukan tugas tertentu. Tugas yang dapat dilakukan oleh ML pun sangat beragam, tergantung dari apa yang ia pelajari [2].

B.3 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) atau dikenal juga dengan analisis data eksploratif merupakan pendekatan analisis untuk suatu data untuk membuat gambaran keseluruhan (summary) data, sehingga mudah untuk dipahami. Metode analisis ini menyediakan berbagai alat untuk meringkas dan memperoleh wawasan tentang sekumpulan data dengan cepat menggunakan grafik sebagai bentuk visualisasi data, tanpa menggunakan model statistik, atau formulasi hipotesis [3].

B.4 Data Preprocessing

Data preprocessing adalah proses mempersiapkan data mentah agar siap digunakan dalam analisis atau model machine learning. Tahapan ini mencakup pembersihan data, transformasi, integrasi, dan reduksi untuk memastikan data berkualitas tinggi, bebas dari noise, serta dalam format yang sesuai. Dengan preprocessing yang baik, model dapat belajar lebih efektif, menghindari bias, dan menghasilkan prediksi yang lebih akurat [4].

C Hasil dan Pembahasan

C.1 Exploratory Data Analysis (EDA)

C.1.1 Import Library

Ada 3 library yang digunakan untuk melakukan EDA, yaitu Pandas, Matplotlib, dan Seaborn. Pandas digunakan untuk memanipulasi data, Matplotlib digunakan untuk membuat visualisasi data, dan Seaborn digunakan untuk membuat visualisasi data yang lebih kompleks dan menarik.

Pada pertemuan sebelumnya, praktikan telah menginstall library pandas dan numpy. Namun, untuk library matplotlib, praktikan perlu menginstallnya terlebih dahulu menggunakan perintah:

```
pip install matplotlib
```

Berikut adalah kode untuk mengimpor library yang digunakan dalam EDA:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

C.1.2 Load Dataset

Dataset yang akan digunakan di dalam praktikum ini diambil dari kaggle dengan link `https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers` Untuk membaca dataset, praktikan dapat menggunakan perintah:

```
df_churning =
   pd.read_csv('https://raw.githubusercontent.com/ganjar87
/data_science_practice/main/BankChurners.csv',
   delimiter=',')
```

C.1.3 Melihat 5 baris pertama dataset

Untuk melihat 5 baris pertama dataset, praktikan dapat menggunakan perintah:

```
df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/BankChurners.csv', delimiter=',')
df_churning.head()
```

✓ 0.8s

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Mo
0	768805383	Existing Customer	45	M	3	High School	Married	$60K - $80K	Blue	
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than $40K	Blue	
2	713982108	Existing Customer	51	M	3	Graduate	Married	$80K - $120K	Blue	
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than $40K	Blue	
4	709106358	Existing Customer	40	M	3	Uneducated	Married	$60K - $80K	Blue	

5 rows × 21 columns

Gambar 1: Membuat array dengan string sebagai member

C.1.4 Melihat 5 baris terakhir dataset

Untuk melihat 5 baris terakhir dataset, praktikan dapat menggunakan perintah:

```
df_churning.tail()
```

✓ 0.0s

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	I
10122	772366833	Existing Customer	50	M	2	Graduate	Single	
10123	710638233	Attrited Customer	41	M	2	Unknown	Divorced	
10124	716506083	Attrited Customer	44	F	1	High School	Married	
10125	717406983	Attrited Customer	30	M	2	Graduate	Unknown	
10126	714337233	Attrited Customer	43	F	2	Graduate	Married	

5 rows × 21 columns

Gambar 2: Membuat array dengan string sebagai member

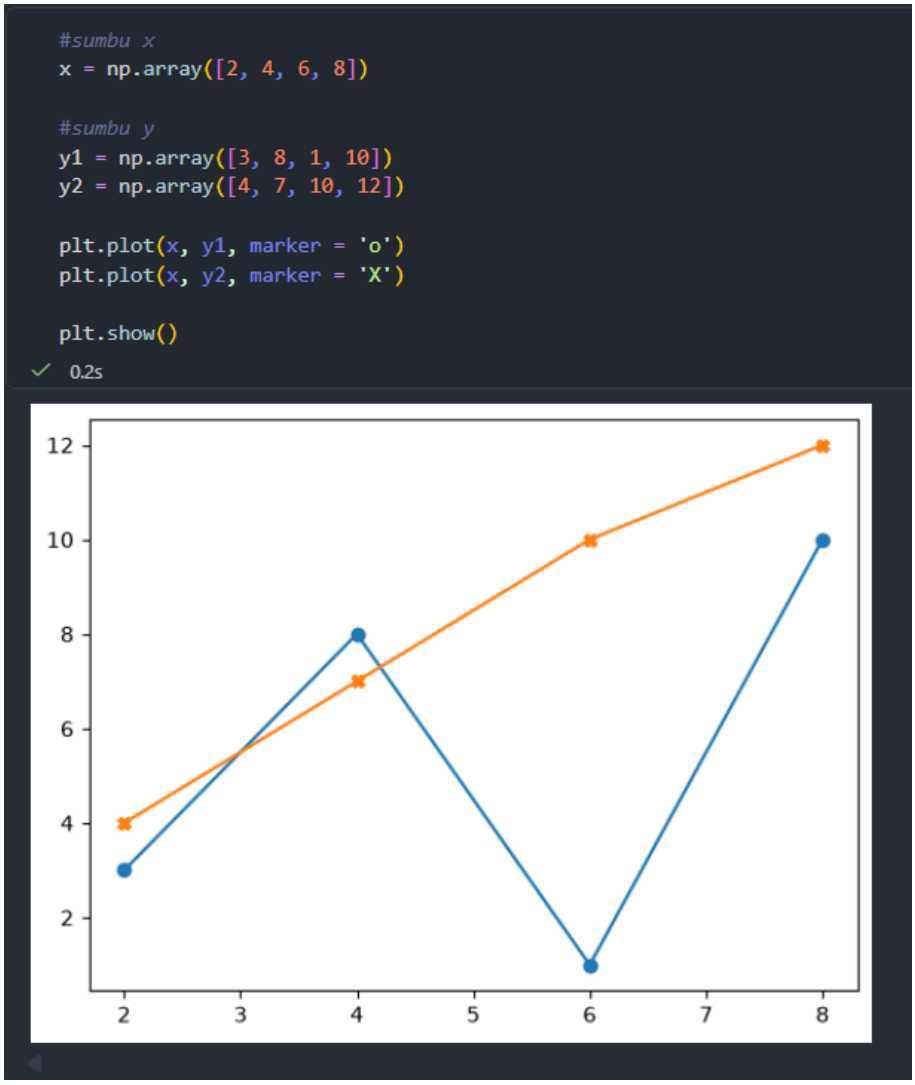
C.1.5 Menyimpan nilai dari class menjadi 2 dataframe yang berbeda

```
existing_data = df_churning[(df_churning.Attrition_Flag ==
    'Existing Customer')]
attrited_data = df_churning[(df_churning.Attrition_Flag ==
    'Attrited Customer')]
```

Kode tersebut digunakan untuk menyimpan nilai dari class menjadi 2 dataframe yang berbeda, yaitu existing_data dan attrited_data. Dataframe existing_data berisi data pelanggan yang masih aktif, sedangkan dataframe attrited_data berisi data pelanggan yang sudah tidak aktif (churn).

C.1.6 Line Chart

Untuk membuat line chart, praktikan dapat menggunakan data dummy terlebih dahulu seperti pada gambar 3.

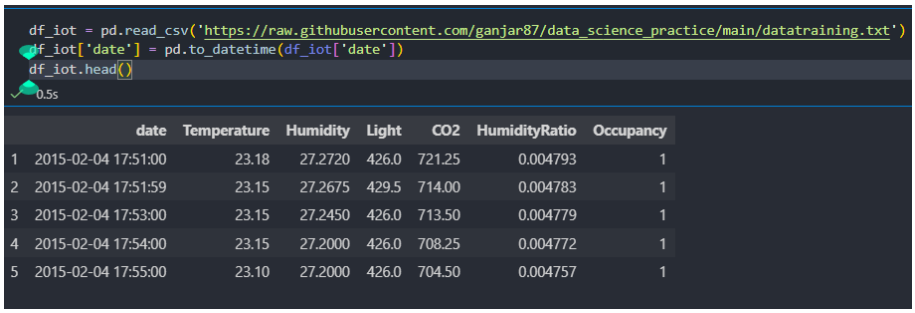


Gambar 3: Linechart dari data dummy

Berdasarkan gambar 3, Kode tersebut digunakan untuk membuat grafik garis

dengan dua data menggunakan library *Matplotlib*. Variabel x berisi data untuk **sumbu** x, sedangkan y_1 dan y_2 berisi data untuk **sumbu** y. Perintah `plt.plot(x, y1, marker='o')` digunakan untuk membuat garis pertama dengan penanda berbentuk lingkaran, sedangkan `plt.plot(x, y2, marker='X')` digunakan untuk membuat garis kedua dengan penanda berbentuk X. Terakhir, `plt.show()` digunakan untuk menampilkan grafik yang telah dibuat.

Setelah mencoba membuat line chart dengan data dummy, praktikan dapat mencoba membuat line chart dengan data yang ada pada dataset ioT untuk menampilkan perubahan nilai sensor terhadap waktu.



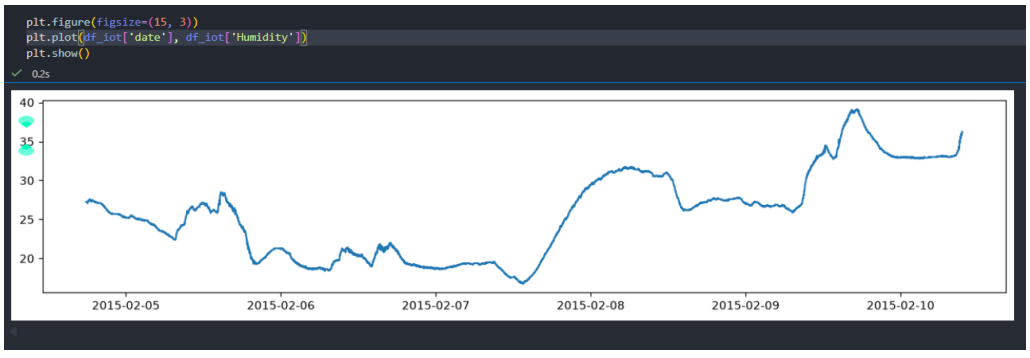
```
df_iot = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/datatraining.txt')
df_iot['date'] = pd.to_datetime(df_iot['date'])
df_iot.head()
```

0.5s

	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

Gambar 4: Load dataset IoT

Selanjutnya, praktikan dapat membuat line chart dengan data yang ada pada dataset ioT untuk menampilkan perubahan nilai sensor terhadap waktu.



Gambar 5: Linechart dari data ioT

Kode tersebut digunakan untuk membuat grafik garis yang menampilkan hubungan antara tanggal dan tingkat kelembapan pada data IoT. Perintah `plt.figure(figsize=(15,`

3)) digunakan untuk mengatur ukuran grafik dengan lebar 15 dan tinggi 3. Selanjutnya, `plt.plot(df_iot['date'], df_iot['Humidity'])` digunakan untuk membuat grafik dengan kolom `date` sebagai sumbu x dan kolom `Humidity` sebagai sumbu y. Terakhir, `plt.show()` digunakan untuk menampilkan grafik yang telah dibuat.

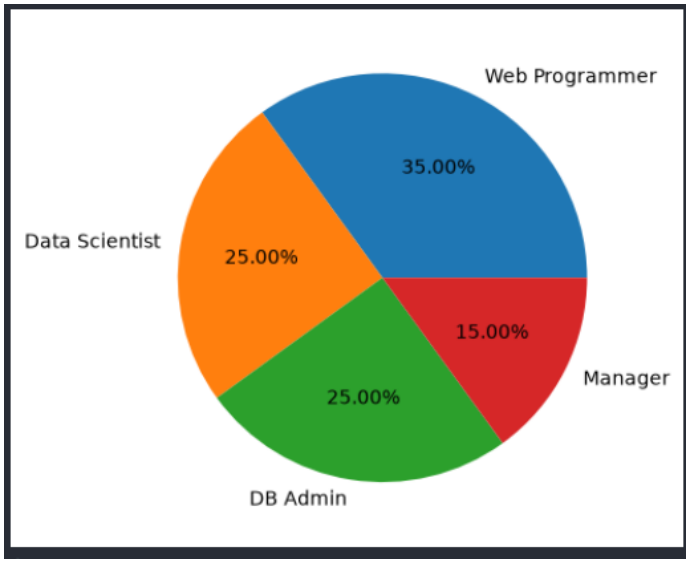
C.1.7 Pie Chart

Pie Chart digunakan untuk menampilkan proporsi dari setiap kategori dalam bentuk lingkaran. Setiap irisan pada lingkaran mewakili persentase dari kategori tersebut. Pie Chart sangat berguna untuk memahami distribusi data secara visual.

```
y = np.array([35, 25, 25, 15])
mylabels = ["Web Programmer", "Data Scientist", "DB
    Admin", "Manager"]

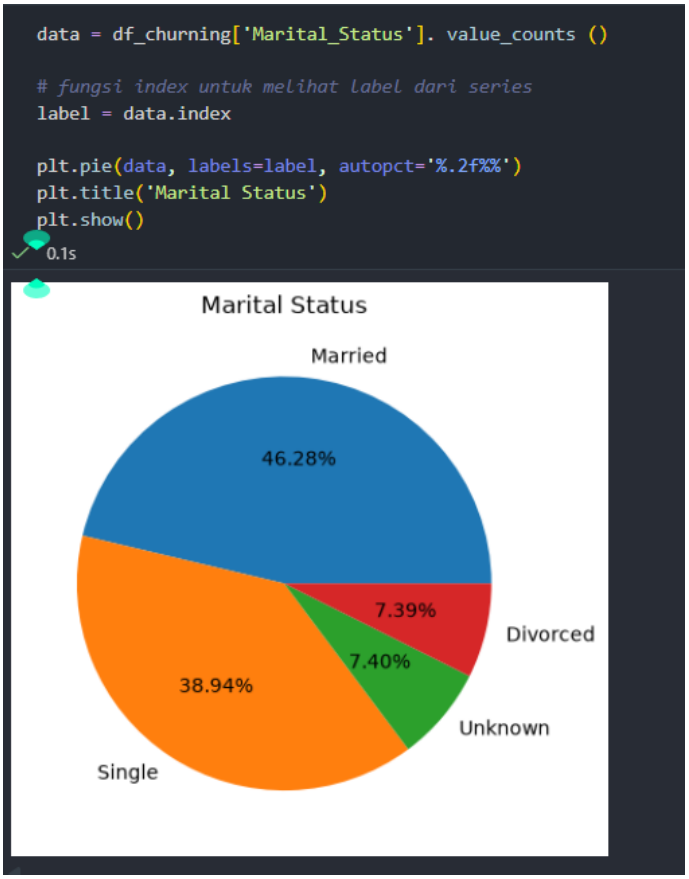
plt.pie(y, labels = mylabels, autopct='%.2f%%')
plt.show()
```

Kode tersebut digunakan untuk membuat *pie chart* yang menunjukkan persentase dari beberapa jenis pekerjaan. Variabel y berisi nilai data, sedangkan `mylabels` berisi label untuk setiap bagian, yaitu *Web Programmer*, *Data Scientist*, *DB Admin*, dan *Manager*. Perintah `plt.pie(y, labels=mylabels, autopct='%.2f%%')` digunakan untuk membuat diagram lingkaran dan menampilkan persentase setiap kategori dengan dua angka di belakang koma. Terakhir, `plt.show()` digunakan untuk menampilkan diagram seperti pada gambar 6.



Gambar 6: Pie Chart dari Data Pekerjaan

Selanjutnya, pembuatan pie chart dilakukan menggunakan dataset sebelumnya yaitu Bank Churner. Visualisasi ini bertujuan untuk menampilkan proporsi kategori pada variabel Marital_Status.



Gambar 7: Pie Chart dari Data Marital Status

Kode tersebut digunakan untuk membuat *pie chart* yang menampilkan distribusi status pernikahan dari data pelanggan. Perintah `df_churning['Marital_Status'].value_counts()` digunakan untuk menghitung jumlah data pada setiap kategori status pernikahan. Selanjutnya, `data.index` digunakan untuk mengambil label atau nama kategori dari *Series*. Perintah `plt.pie(data, labels=label, autopct='%.2f%')` digunakan untuk membuat diagram lingkaran dan menampilkan persentase setiap kategori dengan dua angka di belakang koma. Perintah `plt.title('Marital Status')` memberikan judul pada grafik, sedangkan `plt.show()` digunakan untuk menampilkan grafik.

C.1.8 Bar Plot

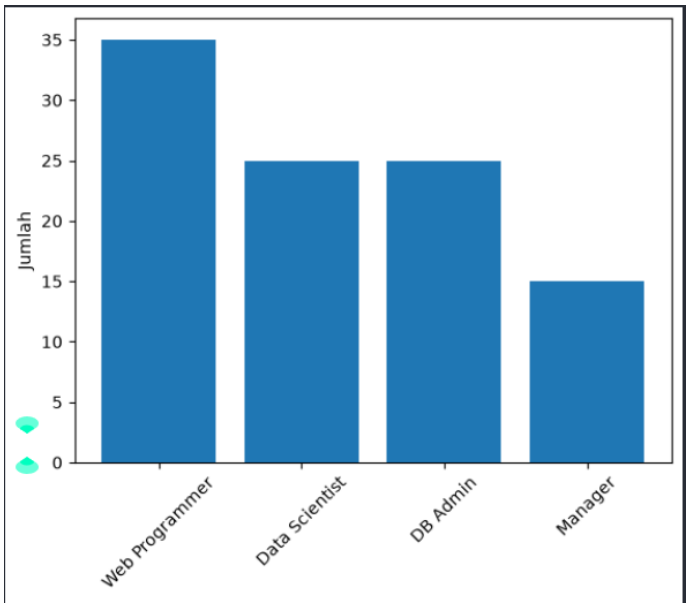
```
y = np.array([35, 25, 25, 15])
```

```
x = ["Web Programmer", "Data Scientist", "DB Admin",
    "Manager"]

plt.bar(x,y)

plt.xticks(rotation=45)
plt.ylabel('Jumlah')
plt.show()
```

Kode tersebut digunakan untuk membuat *bar chart* yang menampilkan jumlah pada setiap kategori pekerjaan. Variabel y berisi nilai jumlah, sedangkan x berisi nama kategori. Perintah `plt.bar(x,y)` membuat grafik batang, `plt.xticks(rotation=45)` memiringkan label sumbu x sebesar 45°, dan `plt.ylabel('Jumlah')` memberikan label pada sumbu y. Terakhir, `plt.show()` digunakan untuk menampilkan grafik seperti pada gambar 8.



Gambar 8: Bar Plot dari Data Pekerjaan

Selanjutnya, buat plot dengan dataset bank churner untuk membandingkan distribusi kategori marital_status antara 2 kelompok nasabah, yaitu existing customer

dan attrited customer. Visualisasi ini bertujuan untuk menampilkan perbandingan distribusi kategori pada variabel Marital_Status antara kedua kelompok nasabah.

```
y1 = existing_data['Marital_Status'].value_counts()
y2 = attrited_data['Marital_Status'].value_counts()

x1 = np.arange(len(y1.index))
x2 = np.arange(len(y2.index))
bar_width = 0.4

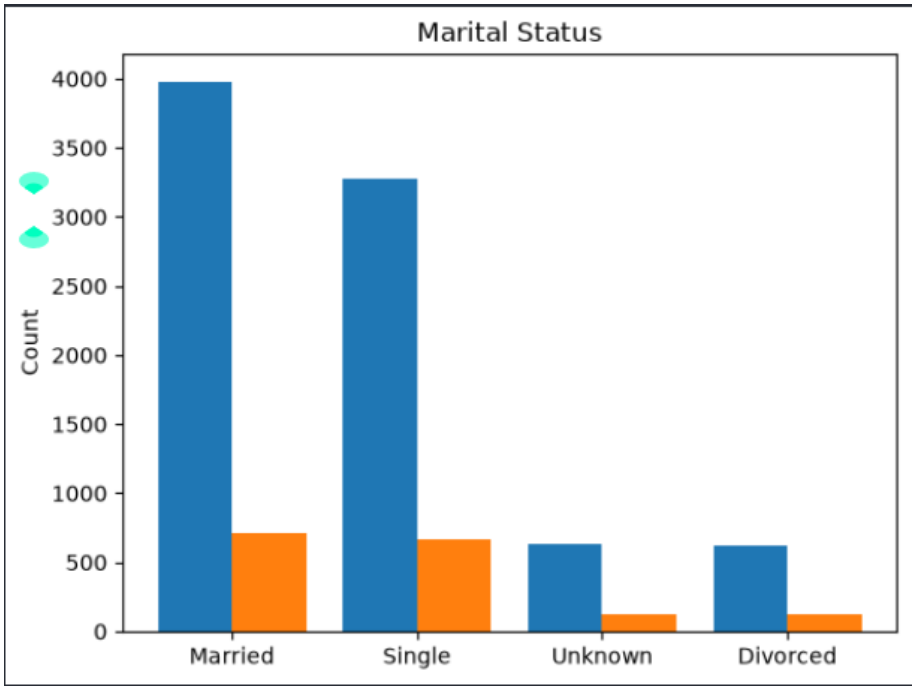
plt.bar(x1, y1, width=bar_width)
plt.bar(x2+bar_width, y2, width=bar_width)

plt.title('Marital Status')
plt.ylabel('Count')
plt.xticks(x2 + bar_width / 2, labels=df_churning.Marital_Status.unique())

plt.show()
✓ 0.2s
```

Gambar 9: Perbandingan Distribusi Marital Status

Kode tersebut digunakan untuk membuat *bar chart* yang membandingkan jumlah pelanggan berdasarkan status pernikahan pada dua kelompok data, yaitu *existing* dan *attrited*. Variabel y_1 dan y_2 digunakan untuk menghitung jumlah setiap kategori `Marital_Status`, sedangkan x_1 dan x_2 menentukan posisi batang pada sumbu x. Variabel `bar_width` mengatur lebar batang sebesar 0.4. Dua perintah `plt.bar()` digunakan untuk menampilkan kedua kelompok data secara berdampingan. Selanjutnya, `plt.title()` memberikan judul grafik, `plt.ylabel()` memberikan label sumbu y, dan `plt.xticks()` memberikan label kategori pada sumbu x. Terakhir, `plt.show()` digunakan untuk menampilkan grafik.



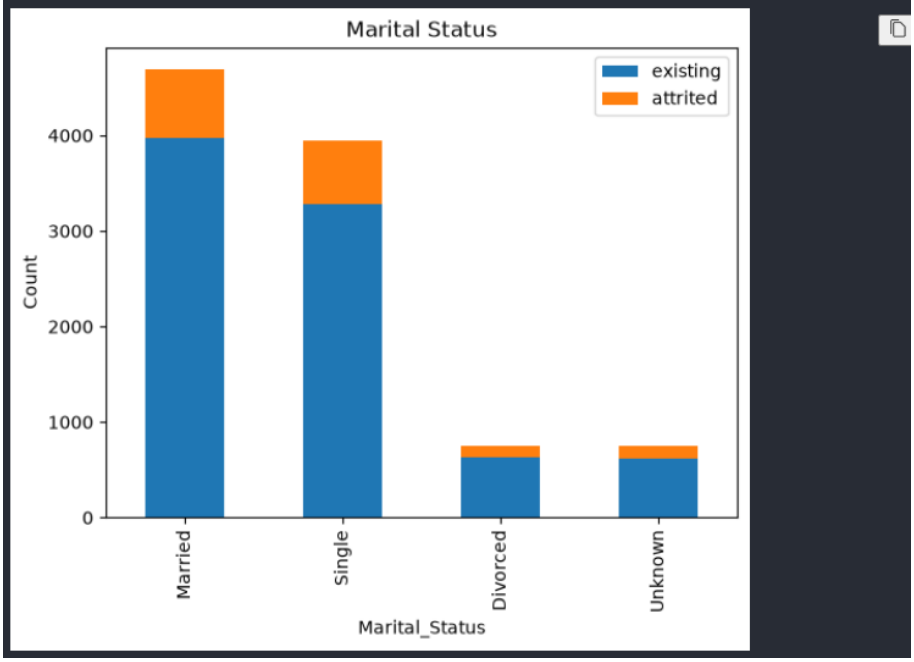
Gambar 10: Perbandingan Distribusi Marital Status

C.1.9 Stacked Bar Plot

Stacked bar plot adalah jenis grafik batang yang digunakan untuk menampilkan distribusi data dalam bentuk tumpukan. Setiap batang mewakili total nilai dari beberapa kategori, dan setiap kategori ditumpuk di atas satu sama lain. Stacked bar plot sangat berguna untuk membandingkan proporsi kategori dalam satu kelompok data. Berikut merupakan contoh kode untuk membuat stacked bar plot menggunakan dataset Bank Churner untuk membandingkan distribusi kategori Marital_Status antara 2 kelompok nasabah, yaitu existing customer dan attrited customer.

```
# menggunakan dataset diatas
# contoh menggunakan stacked bar plot
data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()
df_new = pd.concat([data1, data2],keys=['existing', 'attrited'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', stacked=True) # bisa pakai cara 2
plt.title('Marital Status')
plt.ylabel('Count')
plt.show()
```



Gambar 11: Stacked Bar Plot dari Data Marital Status

Kode tersebut digunakan untuk membuat *stacked bar plot* yang membandingkan jumlah pelanggan *existing* dan *attrited* berdasarkan `Marital_Status`. Perintah `value_counts()` menghitung jumlah tiap kategori, kemudian `pd.concat()` menggabungkan kedua data tersebut. Selanjutnya, `plot(kind='bar', stacked=True)` membuat diagram batang bertumpuk. Perintah `plt.title()` dan `plt.ylabel()` digunakan untuk memberikan judul dan label sumbu y, sedangkan `plt.show()` menampilkan grafik.

C.1.10 Histogram

Histogram adalah jenis grafik yang digunakan untuk menampilkan distribusi data numerik dalam bentuk batang. Setiap batang mewakili jumlah data yang berada

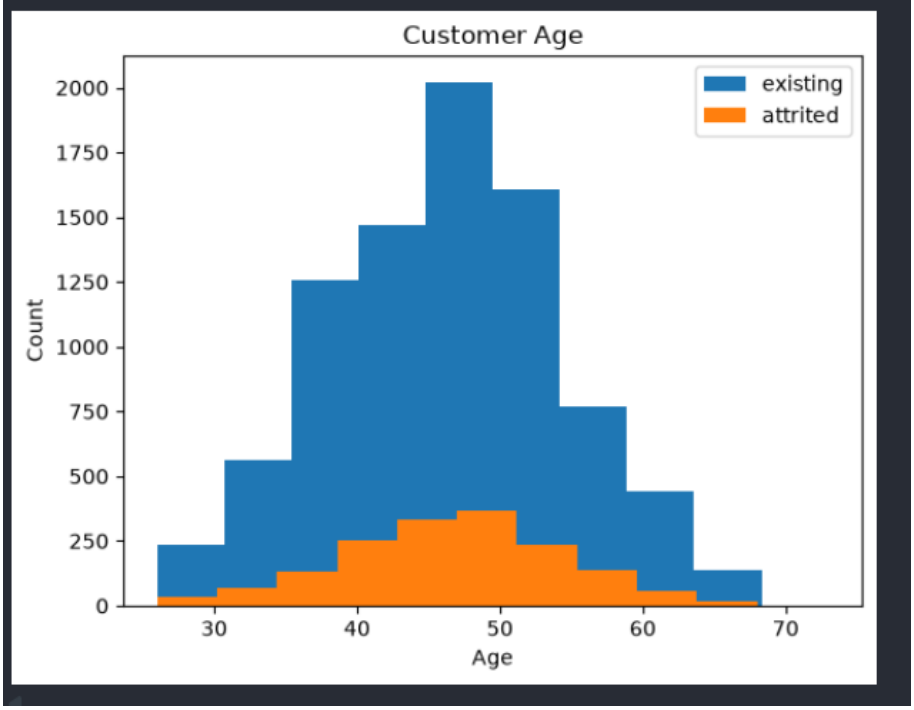
dalam rentang nilai tertentu (bin). Histogram sangat berguna untuk memahami sebaran data, mengidentifikasi pola, dan mendeteksi outlier. Berikut merupakan contoh kode untuk membuat histogram menggunakan dataset Bank Churner untuk menampilkan distribusi variabel Customer_Age.

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

plt.hist(data1, label='existing')
plt.hist(data2, label='attrited')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend()
plt.show()
```



Gambar 12: Histogram dari Data Customer Age

Kode tersebut digunakan untuk membuat *histogram* yang membandingkan distribusi usia pelanggan pada kelompok *existing* dan *attrited*. Variabel `data1` dan `data2` mengambil data usia pelanggan dari masing-masing kelompok. Perintah `plt.hist()` digunakan untuk membuat histogram, sedangkan `plt.title()`, `plt.ylabel()`,

dan `plt.xlabel()` digunakan untuk memberikan judul serta label pada sumbu grafik. Perintah `plt.legend()` menampilkan keterangan untuk membedakan kedua kelompok, dan `plt.show()` digunakan untuk menampilkan grafik.

C.1.11 Scatter Plot

Pembuatan scatter plot dilakukan untuk menampilkan hubungan antara dua variabel numerik. Scatter plot sangat berguna untuk mengidentifikasi pola, tren, dan korelasi antara variabel. Berikut merupakan contoh kode untuk membuat scatter plot menggunakan dataset Bank Churner dengan plt.scatter().

```
y1 = attrited_data['Months_on_book']
x1 = attrited_data['Customer_Age']

y2 = existing_data['Months_on_book']
x2 = existing_data['Customer_Age']

plt.scatter(x2, y2)
plt.scatter(x1, y1)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')

plt.show()
```

Gambar 13: Pembuatan Scatter Plot dengan plt.scatter



Gambar 14: Scatter Plot dari Data Customer Age dan Credit Limit

Versi lain dalam pembuatan scatter plot menggunakan .plot.scatter() yang lebih ringkas dan terstruktur.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.scatter dari object dataframe
ax=existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')
attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')
plt.show()
```

Gambar 15: Pembuatan Scatter Plot dengan plt.plot



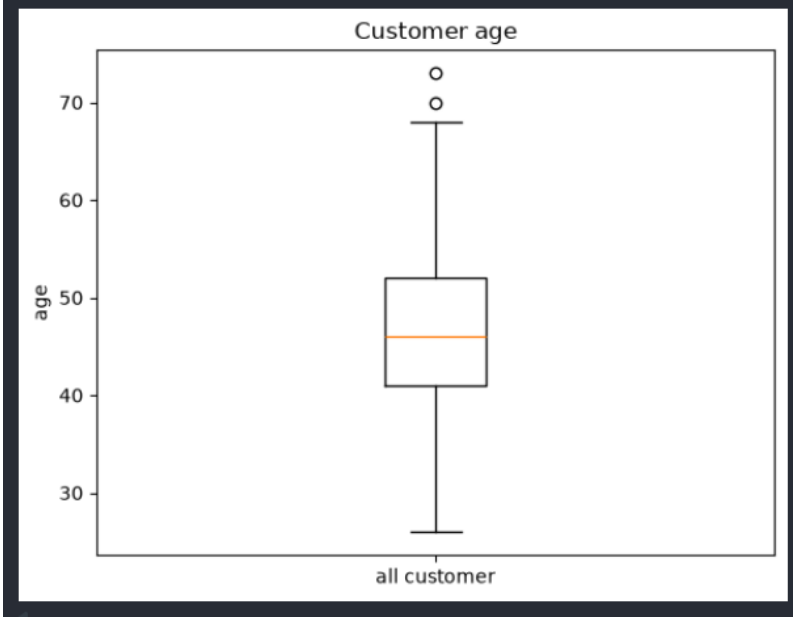
Gambar 16: Scatter Plot dari Data Customer Age dan Credit Limit

C.1.12 Box Plot

Box plot adalah jenis grafik yang digunakan untuk menampilkan distribusi data numerik melalui lima nilai ringkasan: minimum, kuartil pertama (Q1), median (Q2), kuartil ketiga (Q3), dan maksimum. Box plot sangat berguna untuk mengidentifikasi outlier dan memahami sebaran data. Berikut merupakan contoh kode untuk membuat box plot menggunakan dataset Bank Churner untuk menampilkan distribusi variabel Customer_Age.

```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi boxplot dari plt. single group.
plt.boxplot(df_churning['Customer_Age'])

plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1], labels=['all customer'])
plt.show()
```



Gambar 17: Box Plot dari Data Customer Age

Selain itu, boxplot juga dapat dibuat dengan menggunakan fungsi plot.box() dari objek dataframe.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.box dari object dataframe
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

df_new.plot.box() #bisa pakai cara ini
#df_new.plot(kind='box') #bisa menggunakan cara ini juga

plt.title('Customer age')
plt.ylabel('age')
plt.show()
```



Gambar 18: Box Plot dari Data Customer Age

C.1.13 Violin Plot

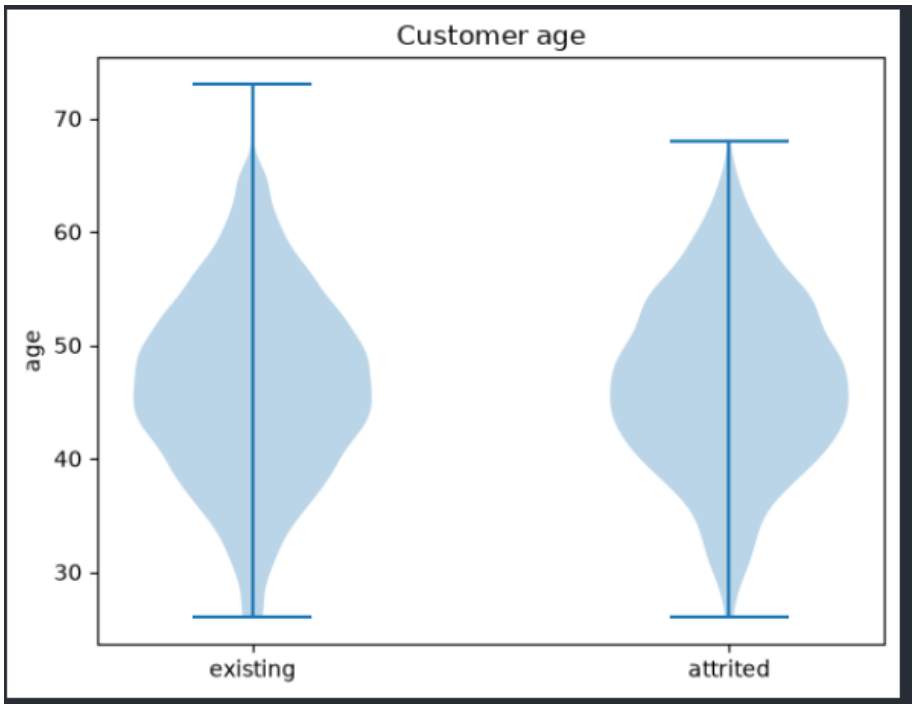
```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi violinplot dari plt. multiple group.

data1 = existing_data['Customer_Age'].values
data2 = attrited_data['Customer_Age'].values
#buat object dictionary dahulu
my_dictionary = {'existing': data1, 'attrited': data2}
#ambil value nya
plt.violinplot(list(my_dictionary.values()))

plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1,2], labels=list(my_dictionary.keys()))
plt.show()
```

Gambar 19: Violin Plot dari Data Customer Age

Kode tersebut digunakan untuk membuat *violin plot* yang membandingkan distribusi usia pelanggan pada kelompok *existing* dan *attrited*. Data usia disimpan dalam *dictionary* dengan nama masing-masing kelompok sebagai *key*. Perintah `plt.violinplot()` digunakan untuk menampilkan distribusi data, sedangkan `plt.xticks()` memberikan label *existing* dan *attrited* pada sumbu x. Perintah `plt.title()` dan `plt.ylabel()` memberikan judul serta label sumbu y, kemudian `plt.show()` digunakan untuk menampilkan grafik.



Gambar 20: Output Violin Plot dari Data Customer Age

C.1.14 Sub Plot

Subplot digunakan untuk menampilkan beberapa grafik dalam satu figure. Hal ini memungkinkan perbandingan visual yang lebih mudah antara berbagai set data atau jenis grafik. Berikut merupakan contoh kode untuk membuat subplot menggunakan dataset Bank Churner.

```
#versi 1
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1) #2 baris, 2 kolom, plot 1
plt.boxplot(existing_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['existing customer'])

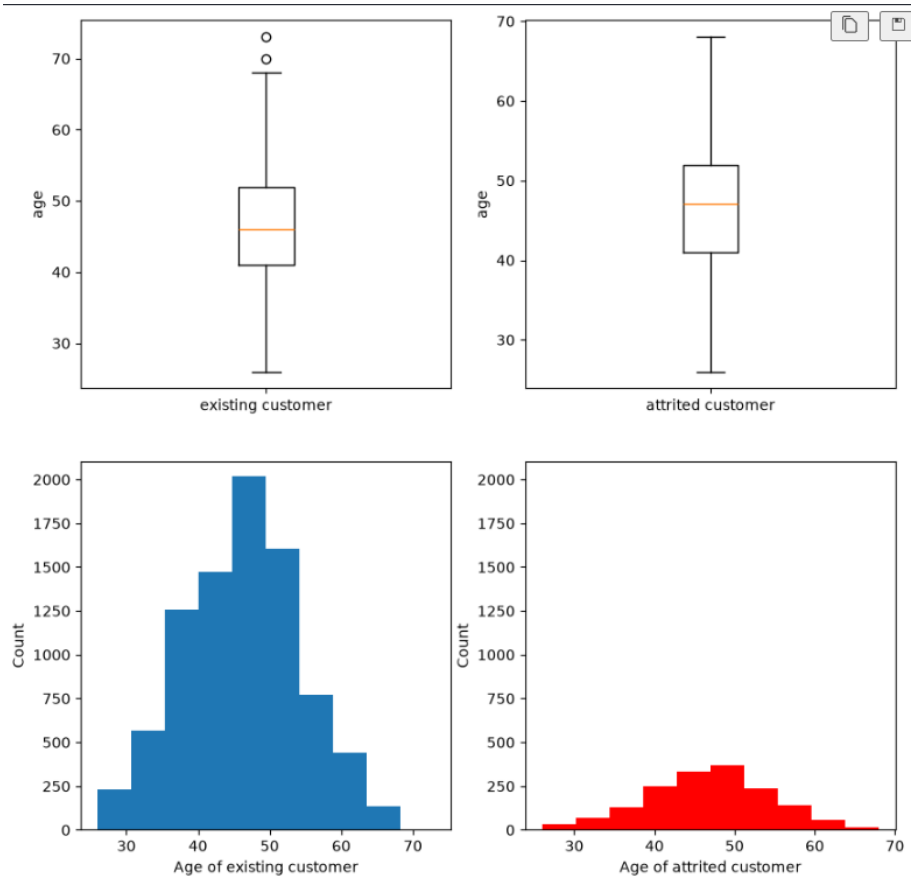
plt.subplot(2, 2, 2)
plt.boxplot(attrited_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['attrited customer'])

plt.subplot(2, 2, 3)
existing_data['Customer_Age'].plot.hist()
plt.ylabel('Count')
plt.xlabel('Age of existing customer')
plt.ylim([0, 2100])

plt.subplot(2, 2, 4)
attrited_data['Customer_Age'].plot.hist(color='r')
plt.ylabel('Count')
plt.xlabel('Age of attrited customer')
plt.ylim([0, 2100])

plt.show()
```

Gambar 21: Sub Plot dari Data Customer Age dan Credit Limit



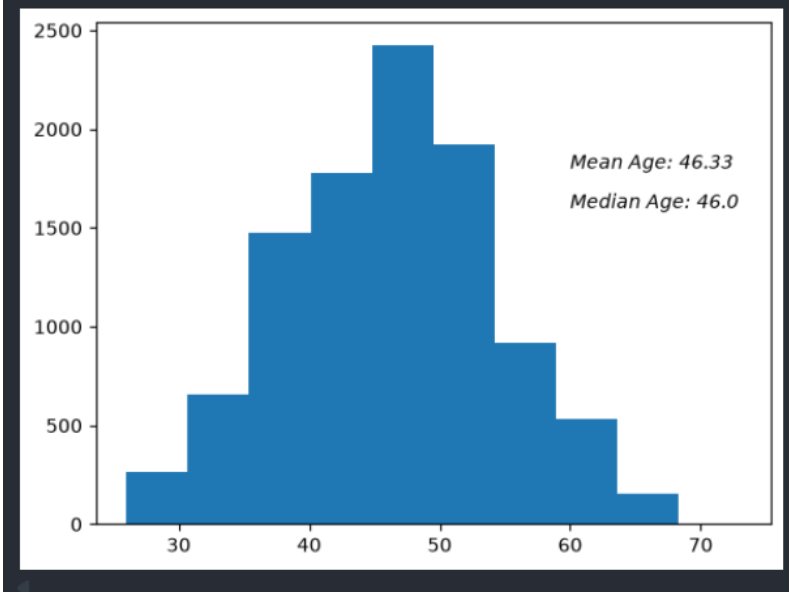
Gambar 22: Output Sub Plot dari Data Customer Age dan Credit Limit

C.1.15 Annotation

Annotation digunakan untuk menambahkan teks atau keterangan pada grafik. Hal ini berguna untuk memberikan informasi tambahan, menyoroti titik tertentu, atau menjelaskan data yang ditampilkan. Berikut merupakan contoh kode untuk menambahkan annotation pada grafik menggunakan dataset Bank Churner.

```
# Show the distribution of a data and its measure of central tendency
data = df_churning['Customer_Age']
plt.hist(data)

plt.text(60, 1600, "Median Age: " + str(round(data.median() , 2)),
style = 'italic', fontsize=10)
plt.text(60, 1800, "Mean Age: " + str(round(data.mean(),2)),
style = 'italic', fontsize=10)
plt. show()
```



Gambar 23: Annotation pada Scatter Plot dari Data Customer Age dan Credit Limit

Kode tersebut digunakan untuk menampilkan distribusi usia pelanggan dalam bentuk *histogram* sekaligus menunjukkan ukuran tendensi sentral berupa *mean* dan *median*. Perintah `data.median()` menghitung nilai median, sedangkan `data.mean()` menghitung nilai rata-rata. Perintah `plt.text()` digunakan untuk menampilkan nilai *median* dan *mean* pada grafik, sementara `round()` membulatkan nilainya menjadi dua angka desimal. Terakhir, `plt.show()` digunakan untuk menampilkan grafik.

C.1.16 Axis

Axis digunakan untuk mengatur sumbu pada grafik, termasuk rentang nilai, label, dan skala. Hal ini penting untuk memastikan grafik mudah dibaca dan interpretatif. Berikut merupakan contoh kode untuk mengatur axis pada grafik meng-

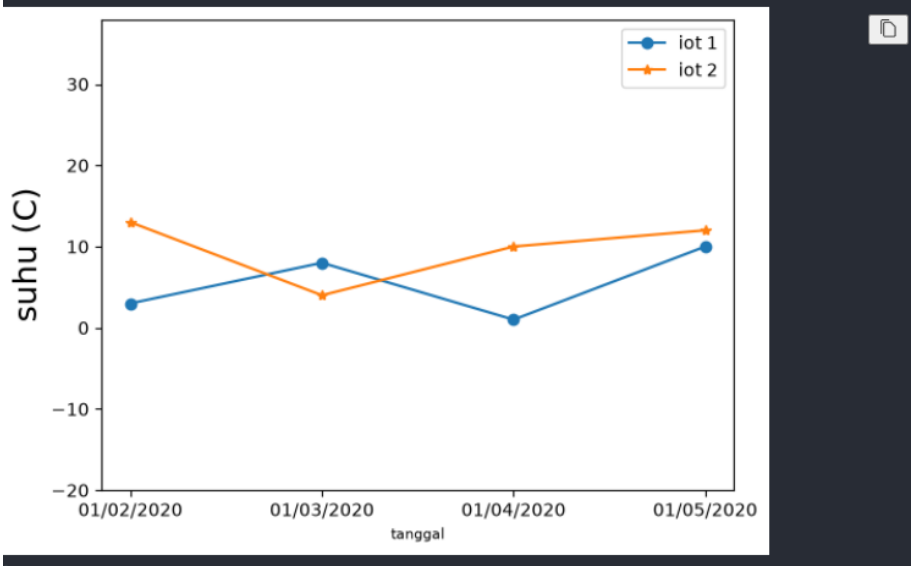
gunakan dataset Bank Churner.

```
#x - np.array([1, 2, 3, 4])
x = np.array(["01/02/2020", "01/03/2020", "01/04/2020", "01/05/2020"])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([13, 4, 10, 12])

plt.plot(x,y1, marker='o', label='iot 1')
plt.plot(x,y2, marker='*', label='iot 2')
plt. legend()

# set Label
plt.ylabel('suhu (C)', fontsize=18) # buat label untuk sumbu y
plt.xlabel('tanggal', fontsize=8) # buat label untuk sumbu x

#set axis range
plt.ylim([-20,38])
plt.show()
```



Gambar 24: Axis pada suhu

Kode tersebut digunakan untuk membuat grafik garis yang membandingkan data suhu dari dua perangkat IoT berdasarkan tanggal. Variabel x berisi tanggal, sedangkan y_1 dan y_2 berisi data suhu dari IoT 1 dan IoT 2. Perintah `plt.plot()` digunakan untuk membuat kedua garis dengan marker yang berbeda, sedangkan `plt.legend()` menampilkan keterangan masing-masing garis. Perintah `plt.xlabel()` dan `plt.ylabel()` memberikan label pada sumbu x dan y, sementara `plt.ylim([-20,38])` mengatur rentang sumbu y dari -20 hingga 38. Terakhir, `plt.show()` digunakan untuk menampilkan grafik.

C.1.17 Legend

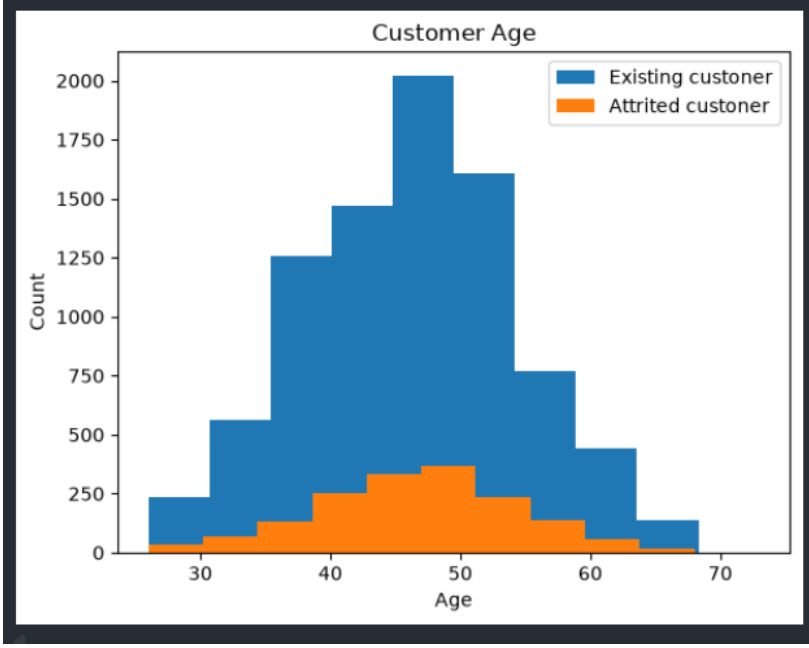
Legend adalah keterangan yang digunakan untuk menjelaskan simbol, warna, atau pola pada grafik. Legend membantu pembaca memahami data yang ditampilkan dan membedakan antara berbagai elemen dalam grafik. Berikut merupakan contoh kode untuk menambahkan legend pada grafik menggunakan dataset Bank Churner.

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

plt.hist(data1, label='Existing customer') # pertama, set label dulu
plt.hist(data2, label='Attrited customer')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend() # kedua, panggil fungsi legend
plt.show()
```



Gambar 25: Legend pada bank churner

Kode tersebut digunakan untuk membuat *histogram* yang membandingkan distribusi usia pelanggan *Existing* dan *Attrited*. Variabel `data1` dan `data2` berisi data usia dari masing-masing kelompok. Perintah `plt.hist()` membuat histogram sekaligus memberikan label pada setiap kelompok, sedangkan `plt.title()`, `plt.ylabel()`,

dan `plt.xlabel()` digunakan untuk memberikan judul serta label pada sumbu grafik. Perintah `plt.legend()` menampilkan keterangan dari kedua kelompok, kemudian `plt.show()` digunakan untuk menampilkan grafik.

C.2 Data Preprocessing

Data preprocessing adalah tahap penting dalam analisis data dan machine learning. Proses ini melibatkan beberapa langkah untuk memastikan data yang digunakan berkualitas tinggi, konsisten, dan siap untuk dianalisis atau digunakan dalam model.

C.2.1 Import Pustaka

Sebelum mengimport pustaka yang penting, praktikan perlu menginstall pustaka scikit-learng terlebih dahulu menggunakan perintah:

```
pip install scikit-learn
```

Setelah berhasil terinstall, import beberapa pustaka seperti berikut:

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

C.2.2 Load Dataset

Praktikan menggunakan dataset dari kaggle dengan link `https://www.kaggle.com/datasets/yasserh/titanic-dataset`. Dataset tersebut kemudian dibaca menggunakan perintah:

```
df =
   pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_
```

C.2.3 Melihat Lima Baris Data Pertama dengan df.head()

```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Gambar 26: Melihat Lima Baris Data Pertama dengan df.head()

C.2.4 Melihat lima baris data terkahir dengan df.tail()

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Gambar 27: Melihat Lima Baris Data Terakhir dengan df.tail()

C.2.5 Melihat bentuk dataset dengan df.shape

```
df.shape
```

```
(891, 12)
```

Gambar 28: Melihat Bentuk Dataset dengan df.shape

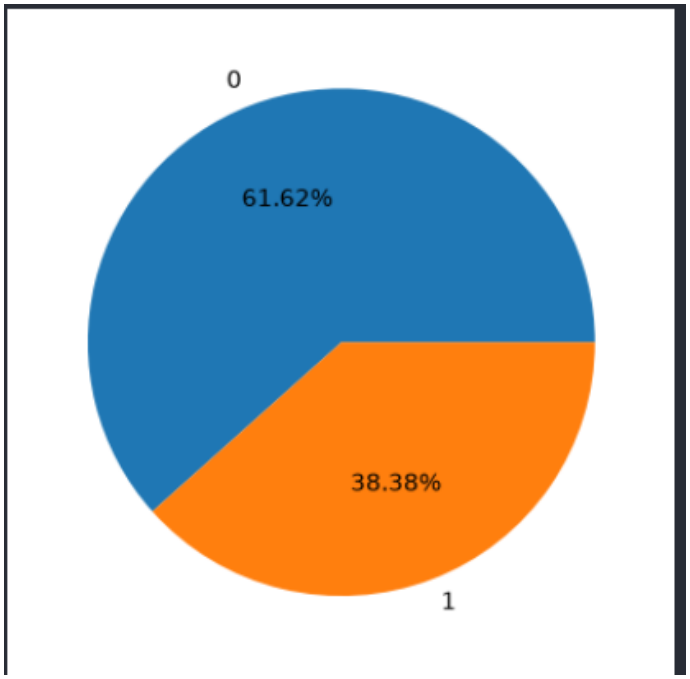
Dapat dilihat pada gambar 28, bahwa terdapat 891 baris dan 12 kolom pada dataset. Hal ini menunjukkan bahwa dataset memiliki 891 sampel data dan 12 fitur atau atribut yang berbeda.

C.2.6 Visualisasi Distribusi Data

```
data = df['Survived'].value_counts()
data.plot(kind='pie', autopct='%.2f%%' )
plt.show()
```

Gambar 29: Visualisasi Distribusi Data dengan Pie Chart

Kode tersebut digunakan untuk membuat *pie chart* yang menunjukkan proporsi data pada kolom `Survived`. Perintah `value_counts()` menghitung jumlah masing-masing kategori, kemudian `data.plot(kind='pie')` membuat diagram lingkaran dengan persentase ditampilkan menggunakan `autopct='%.2f%%'`. Terakhir, `plt.show()` digunakan untuk menampilkan grafik. Hasilnya dapat dilihat pada gambar **??**.

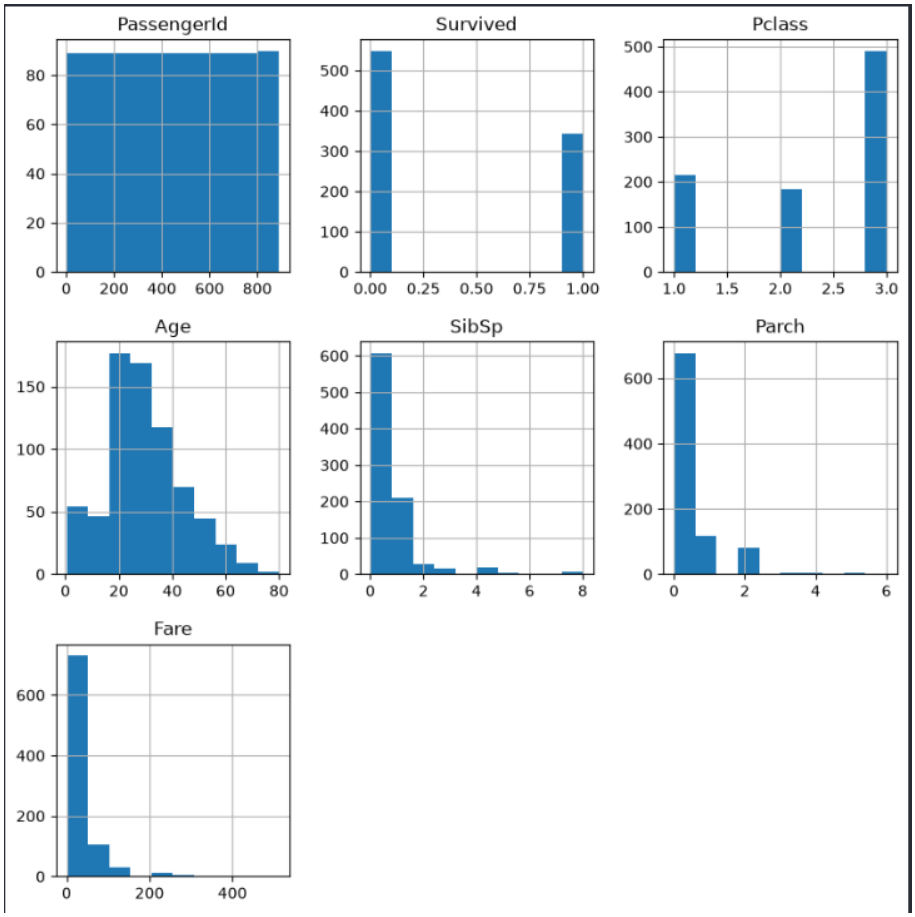


Gambar 30: Output Visualisasi Distribusi Data dengan Pie Chart

Selain itu, praktikan juga dapat membuat visualisasi menggunakan histogram dengan kode berikut:

```
df.hist(figsize=(10,10))
plt.show()
```

Kode di atas menampilkan histogram untuk seluruh kolom numerik dalam dataset guna melihat distribusi datanya. Hasilnya dapat dilihat pada Gambar 31



Gambar 31: Histogram Distribusi Data

C.2.7 Analisis Korelasi antar variabel

```
df.corr(numeric_only=True)
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

Gambar 32: Analisis Korelasi antar Variabel

Kode tersebut digunakan untuk menghitung nilai korelasi antar kolom numerik pada *DataFrame*. Parameter `numeric_only=True` memastikan bahwa hanya data bertipe numerik yang digunakan dalam perhitungan korelasi. Hasilnya berupa matriks korelasi dengan nilai antara -1 hingga 1, yang menunjukkan hubungan antar variabel.

C.2.8 Descriptive Statistics

```
df.describe()
```

Chat (CTRL + I) / Share (CTRL + L)

Pin selection to c

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Gambar 33: Descriptive Statistics dari Dataset

Kode tersebut digunakan untuk menampilkan statistik deskriptif dari data pada *DataFrame*. Fungsi `df.describe()` menghasilkan informasi seperti jumlah data (*count*), rata-rata (*mean*), standar deviasi (*std*), nilai minimum (*min*), kuartil (25%, 50%, 75%), dan nilai maksimum (*max*) untuk kolom numerik.

C.2.9 Mengecek missing values dan duplicates

```
df.isnull().sum()
```

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Gambar 34: Mengecek Missing Values

Berdasarkan gambar 34, Pengecekan missing values menunjukkan bahwa sebagian besar kolom tidak memiliki data kosong, namun terdapat 177 missing pada kolom Age, 687 missing pada Cabin, dan 2 missing pada Embarked. Kolom Cabin, Age dan Embarked perlu di-impute agar dataset tetap dapat digunakan.

```
df.duplicated().sum()
df.drop_duplicates(inplace=True)
```

Kode tersebut digunakan untuk mendeteksi dan menghapus data duplikat pada *DataFrame*. Perintah `df.duplicated().sum()` menghitung jumlah baris yang memiliki data duplikat. Selanjutnya, `df.drop_duplicates(inplace=True)` digunakan untuk menghapus baris duplikat secara langsung pada *DataFrame* asli.

C.2.10 Imputation

```
# Imputation: mengisi nilai kosong

# Kolom numerik
df['Age'] = df['Age'].fillna(df['Age'].median())

# Jika ingin menggunakan mean:
# df['Age'] = df['Age'].fillna(df['Age'].mean())

# Kolom kategori
df['Cabin'] = df['Cabin'].fillna(df['Cabin'].mode()[0])
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

# Cek kembali missing values
df.isnull().sum()
```

Gambar 35: Imputation Data

```
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       0
dtype: int64
```

Gambar 36: Output Imputation Data

C.2.11 Memeriksa categorical attribute

```
# get X and y
df_X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket' ], axis=1)
df_y = df['Survived']

#check categorical attributes
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

```
Index(['Sex', 'Embarked'], dtype='str')
```

Gambar 37: Memeriksa Categorical Attribute

Kode tersebut digunakan untuk memisahkan data menjadi variabel fitur X dan target y. Perintah `df_X` mengambil data fitur dengan menghapus kolom yang tidak digunakan, sedangkan `df_y` mengambil kolom `Survived` sebagai target. Selanjutnya, `select_dtypes()` digunakan untuk mencari kolom pada X yang memiliki tipe data kategorikal, yaitu `object` dan `bool`. Output tersebut menunjukkan bahwa kolom kategorikal dalam fitur X adalah Sex dan Embarked, sehingga kedua kolom ini perlu dilakukan encoding sebelum modelling.

C.2.12 One hot encoding

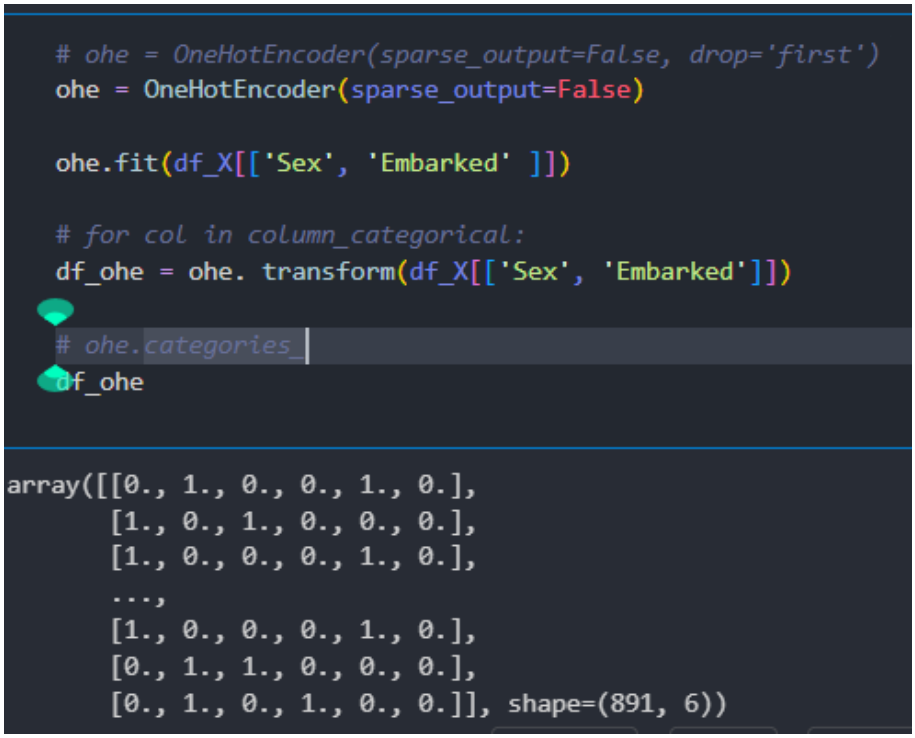
```
# One Hot Encode
df_onehot = pd.get_dummies(df_X, columns=['Sex', 'Embarked'], drop_first=True)
df_onehot. head()
```

	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	True	False	True
1	1	38.0	1	0	71.2833	False	False	False
2	3	26.0	0	0	7.9250	False	False	True
3	1	35.0	1	0	53.1000	False	False	True
4	3	35.0	0	0	8.0500	True	False	True

Gambar 38: One Hot Encoding

Kode tersebut digunakan untuk mengubah data kategorikal pada kolom `Sex` dan `Embarked` menjadi bentuk numerik menggunakan metode *One Hot Encoding*.

Parameter `drop_first=True` digunakan untuk menghapus satu kategori dari setiap variabel agar tidak terjadi redundansi data. Perintah `df_onehot.head()` digunakan untuk menampilkan lima baris pertama dari hasil transformasi.



Gambar 39: Output One Hot Encoding

Kode tersebut digunakan untuk melakukan *One Hot Encoding* pada kolom kategorikal `Sex` dan `Embarked`. Objek `OneHotEncoder` digunakan untuk mempelajari kategori melalui `fit()`, kemudian `transform()` mengubah data kategorikal menjadi bentuk numerik berupa nilai 0 dan 1. Hasil transformasi disimpan dalam variabel `df_ohe` dan kemudian ditampilkan.

C.2.13 Label Encoding

```
#categorical encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
cat_en = LabelEncoder()
for i in cat_features:
    df_X[i] = cat_en.fit_transform(df_X[i])
df_X
```

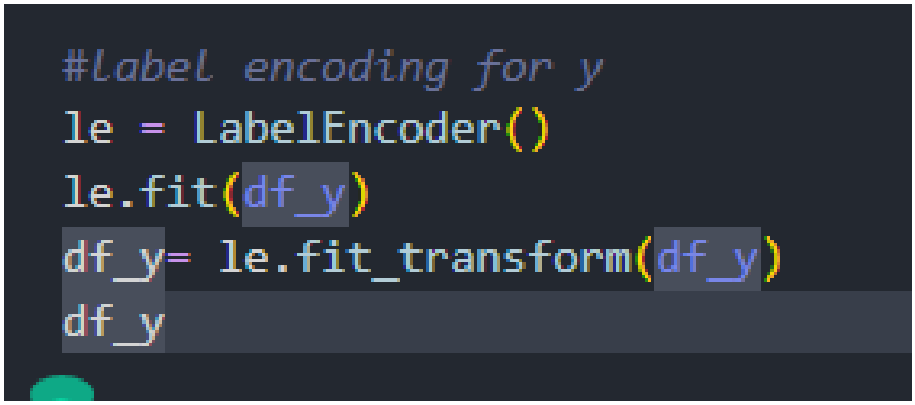
Gambar 40: Label Encoding

Kode tersebut digunakan untuk melakukan *categorical encoding* dengan mengubah data kategorikal menjadi nilai numerik. Perintah `select_dtypes()` digunakan untuk memilih kolom bertipe `object` dan `bool`. Selanjutnya, `LabelEncoder()` mengubah setiap kategori menjadi angka, kemudian proses tersebut dilakukan pada seluruh kolom kategorikal menggunakan *loop*. Hasil akhirnya disimpan kembali ke dalam `df_X`.

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	1	22.0	1	0	7.2500	2
1	1	0	38.0	1	0	71.2833	0
2	3	0	26.0	0	0	7.9250	2
3	1	0	35.0	1	0	53.1000	2
4	3	1	35.0	0	0	8.0500	2
...	...	...	...	...	...	...	...
886	2	1	27.0	0	0	13.0000	2
887	1	0	19.0	0	0	30.0000	2
888	3	0	NaN	1	2	23.4500	2
889	1	1	26.0	0	0	30.0000	0
890	3	1	32.0	0	0	7.7500	1

891 rows × 7 columns

Gambar 41: Output Label Encoding



Gambar 42: Output Label Encoding

Kode tersebut digunakan untuk melakukan *Label Encoding* pada variabel target y. Objek `LabelEncoder()` digunakan untuk mengubah kategori pada `df_y` menjadi nilai numerik. Perintah `fit_transform()` melakukan proses pembelajaran kategori sekaligus mengubahnya menjadi angka. Hasil akhirnya disimpan kembali ke dalam `df_y`.

```
array([0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1,
       1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0,
       0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0,
       1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0,
       0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
       0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0,
       0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0,
       1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1,
...
       0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1,
       1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0])
```

Gambar 43: Output Label Encoding

C.2.14 Mengecek korelasi atribut

```
df_y_new = pd. DataFrame(df_y, columns=[ 'Survived' ])
df_gabung = pd.concat([df_X, df_y_new], axis=1)
df_gabung.corr()
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
Pclass	1.000000	0.131900	-0.369226	0.083081	0.018443	-0.549500	0.157112	-0.338481
Sex	0.131900	1.000000	0.093254	-0.114631	-0.245489	-0.182333	0.104057	-0.543351
Age	-0.369226	0.093254	1.000000	-0.308247	-0.189119	0.096067	-0.025252	-0.077221
SibSp	0.083081	-0.114631	-0.308247	1.000000	0.414838	0.159651	0.066654	-0.035322
Parch	0.018443	-0.245489	-0.189119	0.414838	1.000000	0.216225	0.038322	0.081629
Fare	-0.549500	-0.182333	0.096067	0.159651	0.216225	1.000000	-0.221226	0.257307
Embarked	0.157112	0.104057	-0.025252	0.066654	0.038322	-0.221226	1.000000	-0.163517
Survived	-0.338481	-0.543351	-0.077221	-0.035322	0.081629	0.257307	-0.163517	1.000000

Gambar 44: Mengecek Korelasi Atribut

Kode tersebut digunakan untuk menggabungkan data fitur X dengan variabel target y agar dapat dianalisis korelasinya. Perintah `pd.DataFrame()` mengubah `df_y` menjadi *DataFrame* dengan nama kolom `Survived`. Selanjutnya, `pd.concat()` menggabungkan `df_X` dan `df_y_new` berdasarkan baris. Terakhir, `df_gabung.corr()` digunakan untuk menghitung korelasi antar seluruh variabel numerik.

C.2.15 Membagi dataset

```
#hold out, dibagi menjadi training dan testing set
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.3, random_state=42)
X_train
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
445	1	1	4.0	0	2	81.8583	2
650	3	1	NaN	0	0	7.8958	2
172	3	0	1.0	1	1	11.1333	2
450	2	1	36.0	1	2	27.7500	2
314	2	1	43.0	1	1	26.2500	2
...	...	...	...	...	...	...	...
106	3	0	21.0	0	0	7.6500	2
270	1	1	NaN	0	0	31.0000	2
860	3	1	41.0	2	0	14.1083	2
435	1	0	14.0	1	2	120.0000	2
102	1	1	21.0	0	1	77.2875	2

Gambar 45: Membagi Dataset

Kode tersebut digunakan untuk membagi dataset menjadi *training set* dan *testing set* menggunakan metode *hold out*. Perintah `train_test_split()` membagi 70% data untuk *training* dan 30% untuk *testing* melalui parameter `test_size=0.3`. Parameter `random_state=42` digunakan agar pembagian data tetap konsisten setiap kali kode dijalankan. Hasil pembagian disimpan dalam X_{train}, X_{test}, y_{train}, dan y_{test}.

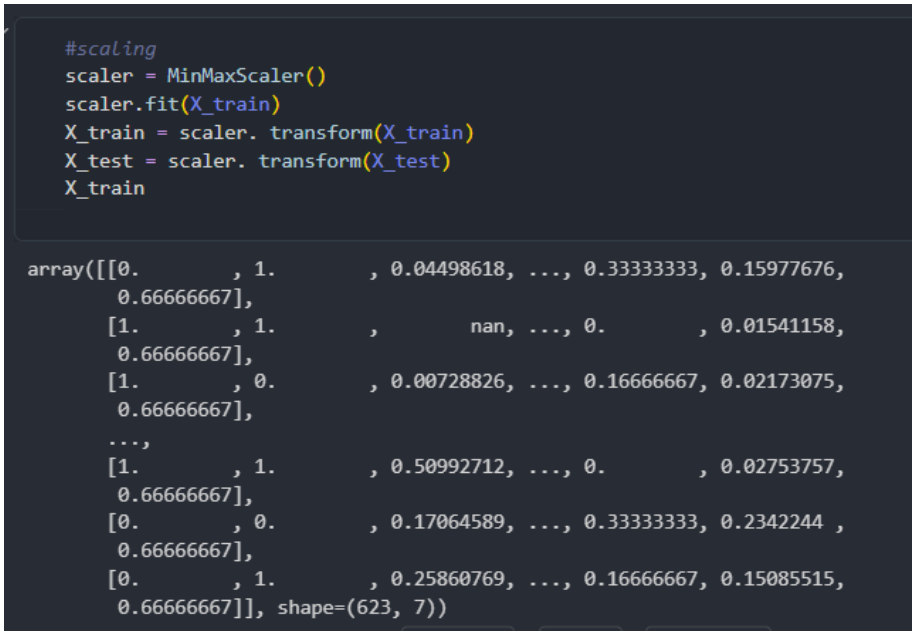
C.2.16 Scaling

```
#scaling
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler. transform(X_train)
X_test = scaler. transform(X_test)
X_train
```

```
array([[-1.63788124,  0.72077194, -1.73655122, ...,  1.99885349,
         0.98099823,  0.56650761],
       [ 0.80326712,  0.72077194,         nan, ..., -0.47932706,
        -0.46963364,  0.56650761],
       [ 0.80326712, -1.38740139, -1.94282224, ...,  0.75976322,
        -0.40613632,  0.56650761],
       ...,
       [ 0.80326712,  0.72077194,  0.80745804, ..., -0.47932706,
        -0.34778742,  0.56650761],
       [-1.63788124, -1.38740139, -1.04898115, ...,  1.99885349,
         1.72907416,  0.56650761],
       [-1.63788124,  0.72077194, -0.5676821 , ...,  0.75976322,
         0.8913508 ,  0.56650761]], shape=(623, 7))
```

Gambar 46: Scaling Data dengan StandardScaler

Kode tersebut digunakan untuk melakukan *scaling* pada data menggunakan `StandardScaler`. Perintah `scaler.fit(X_train)` mempelajari rata-rata dan standar deviasi dari data *training*. Selanjutnya, `transform()` digunakan untuk mengubah X_{train} dan X_{test} ke skala standar, sehingga data memiliki rata-rata mendekati 0 dan standar deviasi mendekati 1.



```
#scaling
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train = scaler. transform(X_train)
X_test = scaler. transform(X_test)
X_train
```

```
array([[0.        , 1.        , 0.04498618, ..., 0.33333333, 0.15977676,
        0.66666667],
       [1.        , 1.        ,        nan, ..., 0.        , 0.01541158,
        0.66666667],
       [1.        , 0.        , 0.00728826, ..., 0.16666667, 0.02173075,
        0.66666667],
       ...,
       [1.        , 1.        , 0.50992712, ..., 0.        , 0.02753757,
        0.66666667],
       [0.        , 0.        , 0.17064589, ..., 0.33333333, 0.2342244 ,
        0.66666667],
       [0.        , 1.        , 0.25860769, ..., 0.16666667, 0.15085515,
        0.66666667]], shape=(623, 7))
```

Gambar 47: Scaling Data dengan MinMaxScaler

Kode tersebut digunakan untuk melakukan *scaling* menggunakan `MinMaxScaler`. Perintah `scaler.fit(X_train)` mempelajari nilai minimum dan maksimum dari data *training*. Selanjutnya, `transform()` mengubah X_{train} dan X_{test} ke dalam skala tertentu, umumnya antara 0 dan 1. Proses ini membantu menyamakan skala antar fitur.

C.3 Tugas dan Analisis

C.3.1 Exploratory Data Analysis

Dataset: https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers

Praktikum Penambangan Data – Teknologi Rekayasa Perangkat Lunak – *2025*

1) Silahkan membaca dataset diatas menggunakan *dataframe*!
2) Jelaskan apa tujuan dari penggunaan dataset ini!
3) Definisikan atribut mana yang menjadi input dan atribut mana yang menjadi output/ class/label!
4) Berikan penjelasan singkat untuk setiap atribut/*variable* dari dataset tersebut

Gambar 48: Soal Exploratory Data Analysis

```python
import pandas as pd
# Membaca dataset
df = pd.read_csv('BankChurners.csv')
# Menampilkan 5 data pertama
df.head()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...
0	768805383	Existing Customer	45	M	3	High School	Married	$60K - $80K	Blue	39	...
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than $40K	Blue	44	...
2	713982108	Existing Customer	51	M	3	Graduate	Married	$80K - $120K	Blue	36	...
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than $40K	Blue	34	...
4	709106358	Existing Customer	40	M	3	Uneducated	Married	$60K - $80K	Blue	21	...

Gambar 49: EDA 1

1. Kode tersebut digunakan untuk membaca dataset dari BankChurners.csv dan menampilkan lima baris pertama dari dataset. Perintah `pd.read_csv()` digunakan untuk membaca file CSV, sedangkan `df.head()` menampilkan lima baris pertama dari data.

2. Tujuan utama dataset ini adalah memprediksi apakah seorang customer akan berhenti menggunakan layanan kartu kredit (churn) atau tetap menjadi cus-

tomer. Permasalahan ini penting bagi bank karena kehilangan customer dapat menyebabkan kehilangan pendapatan. Dengan menganalisis karakteristik dan perilaku customer, bank dapat mengidentifikasi customer yang memiliki kemungkinan tinggi untuk churn dan kemudian melakukan tindakan seperti memberikan penawaran, meningkatkan pelayanan, atau membuat program retensi. Kaggle secara eksplisit menjelaskan bahwa dataset ini dibuat untuk membantu bank memprediksi customer yang kemungkinan akan churn sehingga bank dapat melakukan tindakan preventif.

3. Pada dataset *Credit Card Customers*, atribut yang menjadi **output/class/label** adalah `Attrition_Flag`. Atribut ini menunjukkan status pelanggan, yaitu *Existing Customer* atau *Attrited Customer*. Sementara itu, atribut lainnya digunakan sebagai **input/features** untuk membantu memprediksi status pelanggan tersebut. Dengan demikian, permasalahan ini dapat dikategorikan sebagai klasifikasi, karena output yang diprediksi terdiri dari dua kelas.

4. - `CLIENTNUM`: Nomor identitas unik untuk setiap pelanggan.
   - `Attrition_Flag`: Status pelanggan, yaitu apakah pelanggan masih menggunakan layanan (*Existing Customer*) atau telah berhenti (*Attrited Customer*). Atribut ini merupakan **target/output**.
   - `Customer_Age`: Usia pelanggan dalam tahun.
   - `Gender`: Jenis kelamin pelanggan.
   - `Dependent_count`: Jumlah tanggungan yang dimiliki pelanggan.
   - `Education_Level`: Tingkat pendidikan pelanggan.
   - `Marital_Status`: Status pernikahan pelanggan, seperti menikah, lajang, atau bercerai.
   - `Income_Category`: Kategori pendapatan tahunan pelanggan.

- `Card_Category`: Jenis kartu kredit yang dimiliki pelanggan, seperti Blue, Silver, Gold, atau Platinum.
- `Months_on_book`: Lama pelanggan menjadi nasabah dalam satuan bulan.
- `Total_Relationship_Count`: Jumlah produk atau layanan yang dimiliki pelanggan dari bank.
- `Months_Inactive_12_mon`: Jumlah bulan pelanggan tidak aktif selama 12 bulan terakhir.
- `Contacts_Count_12_mon`: Jumlah interaksi atau kontak pelanggan dengan bank selama 12 bulan terakhir.
- `Credit_Limit`: Batas kredit yang diberikan kepada pelanggan.
- `Total_Revolving_Bal`: Total saldo kredit yang belum dilunasi oleh pelanggan.
- `Avg_Open_To_Buy`: Rata-rata jumlah kredit yang masih tersedia untuk digunakan pelanggan selama 12 bulan terakhir.
- `Total_Amt_Chng_Q4_Q1`: Perubahan jumlah transaksi antara kuartal keempat dan kuartal pertama.
- `Total_Trans_Amt`: Total nilai transaksi pelanggan selama 12 bulan terakhir.
- `Total_Trans_Ct`: Total jumlah transaksi pelanggan selama 12 bulan terakhir.
- `Total_Ct_Chng_Q4_Q1`: Perubahan jumlah transaksi antara kuartal keempat dan kuartal pertama.
- `Avg_Utilization_Ratio`: Rata-rata persentase penggunaan batas kredit pelanggan.
- `Naive_Bayes_Classifier_...`: Nilai hasil prediksi dari model Naive Bayes yang terdapat dalam dataset. Atribut ini sebaiknya tidak digunakan se-

bagai input utama karena berisi hasil klasifikasi yang berkaitan dengan target.

C.3.2 Data Preprocessing

- **Data Preprocessing**

  Dataset: https://www.kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data

  1) Silahkan membaca dataset diatas menggunakan *dataframe*!
  2) Buatlah visualisasi histogram untuk beberapa variabel numerik, lalu jelaskan pola distribusi yang terlihat!
  3) Hitung dan visualisasikan korelasi antar variabel, kemudian jelaskan hubungan yang ditemukan!
  4) Lakukan *exploratory data analysis* (EDA) dengan membuat *descriptive statistics* serta melakukan pengecekan *missing value* dan duplikasi. Jelaskan tindakan apa yang perlu dilakukan jika keduanya ditemukan!
  5) Lakukan *preprocessing* lanjutan berupa *encoding* variabel kategorikal (*one-hot & label encoding*), melakukan *train–test split* (80:20), serta menerapkan *standardization* atau *normalization* pada fitur numerik. Jelaskan alasan penggunaan masing-masing teknik!

Gambar 50: Soal Data Preprocessing

Berikut jawaban dari soal data preprocessing:

1. Untuk membaca dataset dari file CSV, praktikan dapat menggunakan perintah `pd.read_csv()` dari pustaka pandas.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Membaca dataset
df = pd.read_csv('Bengaluru_House_Data.csv')

# Menampilkan 5 data pertama
df.head()
```

✓ 7.1s

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
0	Super built-up Area	19-Dec	Electronic City Phase II	2 BHK	Coomee	1056	2.0	1.0	39.07
1	Plot Area	Ready To Move	Chikka Tirupathi	4 Bedroom	Theanmp	2600	5.0	3.0	120.00
2	Built-up Area	Ready To Move	Uttarahalli	3 BHK	NaN	1440	2.0	3.0	62.00
3	Super built-up Area	Ready To Move	Lingadheeranahalli	3 BHK	Soiewre	1521	3.0	1.0	95.00
4	Super built-up Area	Ready To Move	Kothanur	2 BHK	NaN	1200	2.0	1.0	51.00

Gambar 51: Data Preprocessing 1

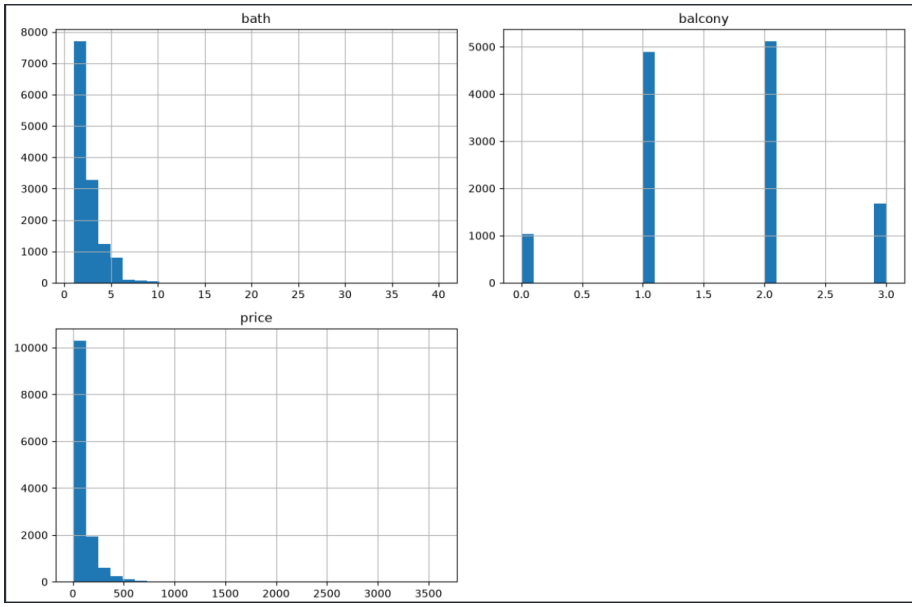
2. Untuk membuat histogram variabel numerik, praktikan dapat menggunakan perintah `df.hist()` dari pustaka pandas. Perintah ini akan menampilkan histogram untuk setiap kolom numerik dalam dataset.

```
df[numeric_cols].hist(
    figsize=(12, 8),
    bins=30
)

plt.tight_layout()
plt.show()
```

Gambar 52: Data Preprocessing 2

Kode tersebut digunakan untuk membuat histogram dari seluruh atribut numerik dalam dataset. Perintah `df[numeric_cols].hist()` membuat histogram untuk setiap kolom numerik dengan ukuran grafik 12×8 dan jumlah *bin* sebanyak 30. Perintah `plt.tight_layout()` digunakan untuk mengatur tata letak agar setiap grafik tidak saling bertumpuk, sedangkan `plt.show()` digunakan untuk menampilkan seluruh histogram.



Gambar 53: Output Histogram Variabel Numerik

3. Untuk memvisualisasikan korelasi antar variabel, berikut yang dapat dilakukan:

```
plt.figure(figsize=(8, 6))

plt.imshow(correlation, cmap='coolwarm', aspect='auto')

plt.colorbar(label='Correlation')

plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=45
)

plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)

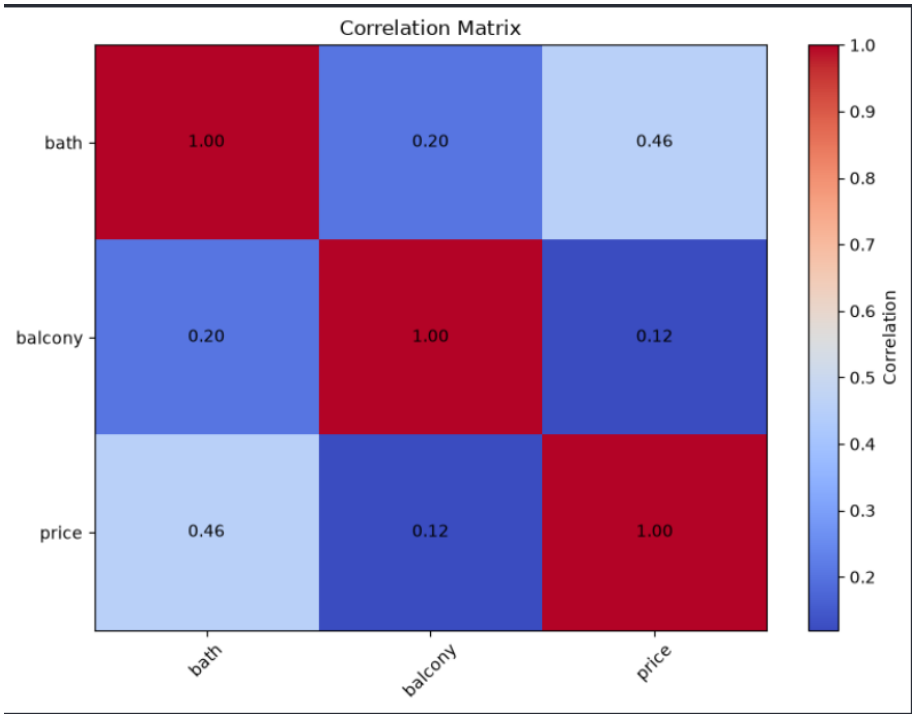
for i in range(len(correlation)):
    for j in range(len(correlation)):
        plt.text(
            j,
            i,
            f'{correlation.iloc[i, j]:.2f}',
            ha='center',
            va='center'
        )

plt.title('Correlation Matrix')

plt.tight_layout()
plt.show()
```

Gambar 54: Visualisasi Korelasi Antar Variabel

Kode tersebut digunakan untuk membuat visualisasi *correlation matrix* dalam bentuk *heatmap*. Perintah `plt.imshow()` menampilkan nilai korelasi dengan gradasi warna, sedangkan `plt.colorbar()` menunjukkan skala korelasi. Perintah `plt.xticks()` dan `plt.yticks()` memberikan nama atribut pada sumbu x dan y. Perulangan `for` digunakan untuk menampilkan nilai korelasi hingga dua angka desimal pada setiap sel. Terakhir, `plt.title()`, `plt.tight_layout()`, dan `plt.show()` digunakan untuk memberikan judul, mengatur tata letak, dan menampilkan grafik.



Gambar 55: Output Visualisasi Korelasi Antar Variabel

4. Melakukan descriptive statistics

```
df.describe(include='all')
```

✓ 0.0s

	area_type	availability	location	size	society	total_sqft	bath	balcony	price
count	13320	13320	13319	13304	7818	13320	13247.000000	12711.000000	13320.000000
unique	4	81	1305	31	2688	2117	NaN	NaN	NaN
top	Super built-up Area	Ready To Move	Whitefield	2 BHK	GrrvaGr	1200	NaN	NaN	NaN
freq	8790	10581	540	5199	80	843	NaN	NaN	NaN
mean	NaN	NaN	NaN	NaN	NaN	NaN	2.692610	1.584376	112.565627
std	NaN	NaN	NaN	NaN	NaN	NaN	1.341458	0.817263	148.971674
min	NaN	NaN	NaN	NaN	NaN	NaN	1.000000	0.000000	8.000000
25%	NaN	NaN	NaN	NaN	NaN	NaN	2.000000	1.000000	50.000000
50%	NaN	NaN	NaN	NaN	NaN	NaN	2.000000	2.000000	72.000000
75%	NaN	NaN	NaN	NaN	NaN	NaN	3.000000	2.000000	120.000000
max	NaN	NaN	NaN	NaN	NaN	NaN	40.000000	3.000000	3600.000000

Gambar 56: Output Descriptive Statistics

Setelah itu, tentukan missing values

```
missing = pd.DataFrame({
    'Jumlah Missing': df.isnull().sum(),
    'Persentase Missing': df.isnull().mean() * 100
})

print(missing)
```

[10] ✓ 0.0s

	Jumlah Missing	Persentase Missing
area_type	0	0.000000
availability	0	0.000000
location	1	0.007508
size	16	0.120120
society	5502	41.306306
total_sqft	0	0.000000
bath	73	0.548048
balcony	609	4.572072
price	0	0.000000

Gambar 57: Output Missing Values

Isi missing value

```
df['bath'] = df['bath'].fillna(
    df['bath'].median()
)
```

✓ 0.0s

```
df['location'] = df['location'].fillna(
    df['location'].mode()[0]
)
```

✓ 0.0s

Gambar 58: Output Imputation

fix duplikasi data

```
df = df.drop_duplicates()
```

✓ 0.0s

```
print("Jumlah duplikasi setelah dihapus:",
      df.duplicated().sum())
```

✓ 0.0s

```
Jumlah duplikasi setelah dihapus: 0
```

Gambar 59: Output Fix Duplicate Data

5. Label Encoding

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

df['area_type_label'] = label_encoder.fit_transform(
    df['area_type']
)

df[['area_type', 'area_type_label']].head()
```

✓ 0.0s

	area_type	area_type_label
0	Super built-up Area	3
1	Plot Area	2
2	Built-up Area	0
3	Super built-up Area	3
4	Super built-up Area	3

Gambar 60: Output Label Encoding

Selanjutnya lakukan standardisasi data dengan StandardScaler

```
        scaler = StandardScaler()

X_train_num = scaler.fit_transform(
    X_train[numeric_cols]
)

X_test_num = scaler.transform(
    X_test[numeric_cols]
)
```

Setelah itu, lakukan train test split dengan perbandingan 80:20

```
        X = df.drop(columns=['price'])
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

D Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa proses eksplorasi data dan visualisasi sangat penting dalam memahami karakteristik dataset. Melalui berbagai jenis grafik seperti bar plot, stacked bar plot, histogram, scatter plot, box plot, violin plot, dan subplot, praktikan dapat mengidentifikasi pola, distribusi, dan hubungan antar variabel. Selain itu, tahap data preprocessing yang meliputi pengecekan missing values, duplikasi data, imputation, encoding, dan scaling sangat krusial untuk memastikan kualitas data sebelum digunakan dalam analisis lebih lanjut atau pemodelan. Dengan pemahaman yang baik terhadap data dan penerapan teknik preprocessing yang tepat, praktikan dapat meningkatkan akurasi model prediksi dan membuat keputusan yang lebih informasional.

E Daftar Pustaka

[1] R. T. Team, "Artificial Intelligence: Pengertian, Sejarah, & Contohnya — ruangguru.com," https://www.ruangguru.com/blog/apa-itu-artificial-intelligence, 2025, [Accessed 28-08-2026].

[2] D. Intern, "Apa itu Machine Learning? Beserta Pengertian dan Cara Kerjanya - Dicoding Blog — dicoding.com," https://www.dicoding.com/blog/machine-learning-adalah/, 2020, [Accessed 28-08-2026].

[3] yunita ws, "MENGENAL EXPLORATORY DATA ANALYSIS – eksplorasidata.mipa.ugm.ac.id — eksplorasidata.mipa.ugm.ac.id," https://eksplorasidata.mipa.ugm.ac.id/2021/08/16/mengenal-exploratory-data-analysis/, 2021, [Accessed 28-08-2026].

[4] D. S. C. Team, "Data Preprocessing: Definisi, Tahapan, dan Implementasinya — digitalskola.com," https://digitalskola.com/blog/data-science/data-preprocessing, 2025, [Accessed 28-08-2026].